Grading Machine-Originated Plant Writes: Separately Inspectable Evidence for Industrial Agent Teams

Anonymous Authors

***Abstract*—Large language model (LLM) agents increasingly propose control changes directly on industrial plants, yet the safeguards surrounding each proposal—write authority, active limits, human approval, device acceptance, process improvement—are rebuilt per deployment inside application code, so a task reported complete can coexist with a missed process objective. AgentWorkShop, a governed supervisory execution layer, carries every agent-originated write of an industrial agent team along one path: admission against declared, versioned limits, approval where the binding requires it, actuation over semantic write-control nodes, graded write acknowledgment by independent post-write read, verdict recording, and compensation—each stage left as a separately inspectable evidence state attributed to its originating actor. Team identities reach data-acquisition (DAQ) and write-control (DCW) nodes only through explicit member–node capabilities, and prompt content is never admissible as authority; a line master switch whose fail-safe default is manual intercepts approval-free recipe dispatch, while protocol drivers, versioned production objects, plugin points, and a typed software development kit (SDK) keep process models, objectives, and policies external to the governed path. Each write is graded readback-verified, transport-acknowledged, or unverified by what an independent read observes, never by the transport receipt alone. Four process scenarios—injection molding, wastewater treatment, annealing, and biaxially oriented polyethylene terephthalate (BOPET) film—and a cast-film reference line exercise the layer across protocol-interception drills, agent-team missions, and a deployment-scale regression campaign on one multi-line instance—intercepting and attributing, grading honestly, pacing automation, mediating approval, and recovering on record; the evaluation is simulation-tier (simulator-backed, not physical plants), and no optimization-benefit or safety-assurance claim is made.**

***Index Terms*—Agent teams, human-in-the-loop, industrial informatics, LLM agents, supervisory control, write governance.**

I. Introduction

LARGE language model (LLM) agents are moving from advising industrial operations to acting on them: recent systems generate control updates for industrial cyber-physical systems from multi-modal input [1], pair each factory module with an LLM agent that dispatches skills over a tool protocol [2], allocate work among cooperating agents [3], [4], and have been studied as real-time controllers of a robot production line [5]. What these demonstrations leave implicit is everything that surrounds the proposal: who may change which parameter, which limits are active for the current product and recipe, whether a human approved, whether the device accepted the command, and whether the process improved.

Because these concerns are normally embedded in application code, they are rebuilt per process, protocol, and agent framework—duplicated drivers, ad-hoc permission checks, logs untied to decisions. The four scenario plants (simulated in this paper) differ—injection molding, wastewater treatment (WWTP), annealing, and BOPET (biaxially oriented polyethylene terephthalate) film—but all need an auditable bridge from a team-level task to heterogeneous acquisition and control nodes. Industrial LLM work to date is mostly generation, diagnosis, and single-domain control (e.g., [6]).

AgentWorkShop addresses this execution gap through member–node bindings (a member being an engine-backed channel identity) and shared line, product, recipe, and run context. The response is organizational rather than algorithmic: each intervention leaves a separate evidence record per governed stage, from admission through compensation (Section IV). Nodes expose engineering quantities rather than register addresses; one governed member–node binding path serves every client, where per-application integration would duplicate drivers, permissions, and logging.

The paper makes four contributions:

1) a **capability-based industrial execution architecture** connecting agent-team identity, semantic data-acquisition (DAQ) and write-control (DCW) nodes, heterogeneous protocol drivers, and line–product–recipe–run context, with task membership distinct from physical-write authority and a default-manual line master switch;
2) a **reusable integration boundary** of self-describing node and driver contracts across six protocol families (five instantiated as lines, four exercised for governed writes), versioned production objects, plugin extension points, and a typed software development kit (SDK), into which an onboarding contract projects scenario blueprints while process models and policies stay external;
3) an **evidence-preserving governed intervention workflow** separating admission, optional approval, actuation, three-level write acknowledgment with post-write verification, verdict, compensation, and backstop—the grading rule stated falsifiably; and
4) a **layered evaluation** exercising one governed path through protocol closure with mechanism ablation, agent-team missions, a deterministic controller loop,

TABLE I
Closest LLM-based industrial control systems versus AgentWorkShop (venue, LLM role, write path, production-scoped authority, write evidence).

System (venue)	Role of the LLM	Plant write path	Authority & production context	Write evidence & verification
MADE, ICCAD 2026 [1]	multi-modal control-update generation	generated commands + safety layer	HITL verification of outputs; task-level checks	task success over scripted evaluations
Köhle et al., ETFA 2026 [2]	per-module agents, MCP skill dispatch	skill invocation via OPC UA/MQTT	module-scoped skills	task-resolution rate (6-module plant)
Lim & Kovalenko, CASE 2025 [3]	capability exploration across agents	architecture-level	resource-matching scope	architecture evaluation
Zhao et al., AEI 2026 [4]	shop-floor task dispatch	workshop experiments	coordination layer	dispatch/execution performance
Waseem et al., Mach. Learn. Eng. 2026 [5]	real-time controller in the loop	direct loop actuation (robot line)	prompt-level constraints	line KPIs under LLM control
AgentWorkShop	**replaceable strategy slot**	**governed write path: admission, HITL, dispatch, readback**	**member–node bindings; line master switch; product/recipe-scoped limits**	**graded acknowledgment; harness-checked three-way consistency; attributed journal; compensation; backstop**

concurrent process missions, and a deployment-scale regression campaign on a multi-line instance (Section V).

Scope (revisited in Section VI): real protocol stacks execute against author-implemented simulators; fast regulation, certified protection, physical-plant transfer, and optimizer ranking are outside the claims.

II. Related Work

A. LLM Agents in Industrial Control: From Generation to Governance

Agent-based manufacturing and supervision have long linked organizational roles to plant capabilities [7], [8]; LLM-oriented industrial work now spans PLC generation and verification [6]. The closest systems, characterized in Table I, target the control action itself: MADE couples multi-modal perception to control generation behind a safety layer, scoring its own task success with no persistent production-scoped write-evidence path reported [1]; Köhle et al. dispatch module-scoped skills over a six-module plant [2]; Waseem et al. place pretrained LLMs inside the control loop as real-time controllers [5]—a layer below AgentWorkShop, so the roles are complementary; Lim and Kovalenko [3] and Zhao et al. [4] allocate work among agents, whereas this layer governs how an allocated proposal reaches the plant and what evidence it leaves. A recent review frames agentic AI's industrial value as safety-aware, explainable human-in-the-loop (HITL) decision support rather than autonomous control authority [9]—the mechanism set contributed here makes that positioning concrete. Table I characterizes each system as reported in its publication.

B. Frameworks, Standards, Run-Time Guarantees, and Automation Practice

Multi-agent frameworks coordinate tasks and tools, and the Model Context Protocol (MCP) standardizes tool and inter-agent interfaces [10]; the tool it exposes, however, is a bare function signature carrying no line, product, or recipe context, with no record separating admission from execution. ISA-88 and ISA-95 supply complementary models for recipes and operations [11], [12]; AgentWorkShop's versioned recipes and run dispatch instantiate a reduced subset of that stack, and the governed life cycle transposes run-to-run (R2R) control disciplines [13] onto a supervisory write path—open-ended proposal space, approval-mediated bindings, and compensation records where R2R has a fitted controller. A separate line enforces run-time guarantees on LLM agents—judge and guard agents, rule-based enforcement [14]; they gate software tool calls, not plant writes, and their policies could compile onto the substrate contributed here.

Permissions, range checks, approvals, and audit trails long existed in distributed-control-system (DCS) and SCADA practice—versioned batch recipes and electronic batch journals under 21 CFR Part 11 and GAMP 5 [15], [16], decoupled second-channel monitoring and advisory operations in NAMUR open architecture [17], and setpoint adaptation in run-to-run control—and IEC 62443 codifies their security counterpart [18], while formal shielding, control-barrier functions, and simplex run-time assurance offer stronger specified properties [19]–[21]. Against that practice the difference is the proposer: a machine-generated agent identity rather than a presupposed human operator, so admission binds to member authority and prompt content stays inert, re-evaluated per request against live product and recipe state rather than at role-assignment time; the governed path spans four exercised write protocol families; and the evidence states—graded acknowledgment, verdict, compensation—survive strategy replacement, where the three-way check is a supervised special case of classical data reconciliation and gross-error detection [22]. AgentWorkShop implements source-dependent checks and a rule-based backstop (Section IV-D) and makes no formal protection guarantee; human oversight likewise requires more than an approval dialog [23]. Agent benchmarks and prompt-injection studies [24], [25] inform adjacent evaluations; ours separates them. Relative to regulated batch and DCS practice—where the operator is presupposed—the increment is admitting a machine-generated proposer into that governed, plant-facing write path, with each write's evidence graded and attributed to a machine actor.

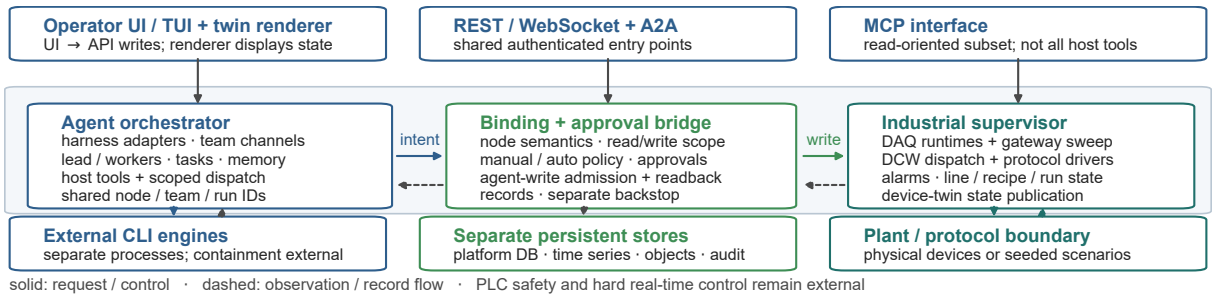


Fig. 1. System architecture and execution boundaries: platform services around shared stores, operator interfaces above, and agent engines, data stores, protocol endpoints, and the plant kept separate. Shared identifiers provide attribution but do not make storage and actuation transactional.

III. System Architecture

AgentWorkShop occupies the replaceable strategy slot that Section II's comparison leaves open: it links team coordination to plant interaction through industrial nodes and production context (Fig. 1). Responsibilities group into five planes—logical groupings, not deployable units: a team plane owns member identity, task state, and episodic memory; an integration plane owns protocol acceptance, readback, freshness, and transport errors, presenting devices as semantic nodes feeding both the agent and a live plant-mirror view; a governance plane owns the governed life cycle—admission through compensation and rollback; an operations plane attaches line, product, recipe, and run context while PLC protection stays independent; and an extensibility plane admits plugins, drivers, agent tools, and SDK clients through sanctioned registration services. Scenario knowledge sits in blueprints, signal maps, predicates, and driver configuration—not in team scheduler, approval service, or persistence code; members are interchangeable across 14 engine adapters, and 118 declared settings plus the typed SDK make team topology, bindings, and limits data rather than code—in every plane, a scenario supplies data and strategy (Fig. 1).

A. Supervisory Core and Nodes

The Node.js core hosts channel routing, industrial services, and protocol adapters. A channel contains a lead, workers, tasks, messages, and retrievable memory, deployed through WebSocket, MCP, agent-to-agent (A2A), REST, CLI, or TUI entry points; channel membership alone confers no write authority.

DAQ nodes convert device readings to engineering quantities across four modalities—scalars, vector profiles, image frames carrying content fingerprints, and mirrored manufacturing-execution-system (MES) records—so plant data-shape heterogeneity does not reach the team loop. MES endpoints are reached in two interchangeable modes—mirror, pulling MES records through the same acquisition pipeline, and direct, declaring read and write mappings against the MES interface. DCW nodes translate engineering setpoints into driver commands and, where supported, readback; per-node runtimes manage timing, and gateway services supply connection state, recipe context, and write bookkeeping. The acquisition surface declares six protocol families, five with drivers today—Modbus TCP, Modbus RTU-over-TCP, OPC UA, MQTT, HTTP/REST, and an S7 stub that fails loudly until its driver lands; the write surface adds one driver per writable family plus MES REST (the evaluation's mock driver lies outside these counts), and the evaluation instantiates five families as lines (Section V-C). The deployment reaches author-implemented dynamic simulators through the same node and driver contracts a plant would use; they are deterministic per seed, not validated digital twins. The first reuse boundary is declarative—signals, units, scaling, precision, and driver configuration—and the team loop is unchanged.

Driver contracts are deliberately narrow: a driver declares how a semantic quantity maps to its transport payload, whether the endpoint is writable, and whether a readback is available, while the controller resolves member authority and active limits. Errors attach to the request, register addresses and byte order appear only here, and a new fieldbus family leaves the loop untouched. Undeclared drivers fail loudly: a node whose driver is not registered is rejected at creation and connection test rather than silently served by a mock. Acquisition is supervisory rather than hard real-time: each DAQ node runs its own cadence under a rotating sweep, and the verdict stage queries the timestamped, quality-stamped history rather than issuing a blocking read.

B. Bindings, Context, and Reuse Boundaries

A member–node binding grants access to a sanctioned tool on a specific node and selects automatic or approval-mediated execution. Line permissions (none, read-only, operate) authorize binding creation and mutation; runtime invocation is authorized by the persisted capability, not channel membership. Intervals are re-checked every request instead of cached, so revoking a binding or switching the recipe takes effect on the next request; the same path serves a console, a harness, and an agent engine, because authority does not depend on the client—only the evidence record differs.

Production objects are versioned rather than overwritten: a recipe is dispatched against a known-good version, reverted non-destructively, and compared with the values actually applied, and an intervention is read against the definition that was active when it was admitted. Plugins

register hooks, routes, agent tools, drivers, and panels only through sanctioned services, so an added driver inherits admission, readback, and evidence capture. Scenario reuse is bounded by an onboarding contract: the engineer declares a blueprint (devices, signals, units, roles, setpoints); a differential "ensure" pass creates missing devices, repairs drift, and projects the blueprint into semantic nodes and a recipe; a channel is built from the team template, and a mission supplies objective, guards, response windows, and policy. A BOPET worker proposes casting-speed, fast-roll, or rail-width changes through bound DCW nodes; the other three scenarios reuse the same pattern (Section V-F). The stores share identifiers, not a transaction domain: a device may accept a write even if bookkeeping fails.

C. Governed Optimization Task Life Cycle

A governed optimization task is a life cycle rather than a single tool call, and AgentWorkShop exposes each transition separately. The task enters a channel carrying goal, line, product, recipe, run, and task identifiers; a lead member claims it and hands it to a worker holding member–node bindings on the relevant nodes, whose fixed job then repeats: read the controlled quantity over a bounded window, propose a setpoint inside the resolved interval, invoke the governed write, wait for the response, and record a verdict (Section V-B instruments it).

Each iteration leaves more than a written value: an optimization record, the acknowledgment grade, an attributed journal entry, predicate attainment, and a workflow terminal state (per-intervention detail in Section IV). Two write lifecycles coexist: direct node writes (agent dcw_control or admin REST) open these records, while recipe-chain operations (batch dispatch) are gated by the run ledger and open none (Section IV). This bounds what "automatic optimization" means here: a controller, a scripted policy, a conventional optimizer, or an agent engine may occupy the strategy slot, while authority, evidence, and closure stay on the platform.

IV. Governed Execution Mechanisms

A supervisory intervention must preserve distinctions that an agent tool call usually collapses—permission to propose, admission of a value, driver acceptance, device confirmation, process response, a caller's verdict, and compensation—and AgentWorkShop exposes these as separate evidence states. Three design commitments generate that separation: model output is treated as an untrusted proposal, never as authority; evidence is graded by what is observed rather than by what was claimed; and failure states are surfaced for inspection rather than absorbed into a success flag. Fig. 2 shows the implemented agent-team runtime that carries these steps; the governance and write stages drawn in the figure are deterministic platform logic—no model output can admit, approve, or confirm a write (Gates 1–2 of Fig. 2). A governed intervention is one transaction in the audit-trail sense:

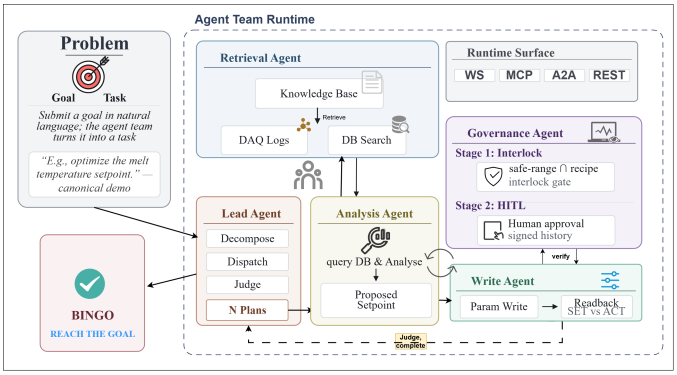


Fig. 2. Implemented agent-team runtime. A natural-language goal enters as a task; the lead agent decomposes and dispatches plans; retrieval and analysis produce the proposed setpoint (the LLM engine occupying a replaceable strategy slot). The governance stages are deterministic platform logic: Gate 1 applies the Eq. (1) interlock, Gate 2 the approval wait; the write agent reads back the value actually present (SET versus observed ACT), and the judge records keep, rollback, or uncertain; every step is journaled with actor attribution. The block labeled "Governance Agent" denotes these deterministic platform stages, not a model-bearing agent; the judge annotation is the caller's verdict-tool invocation.

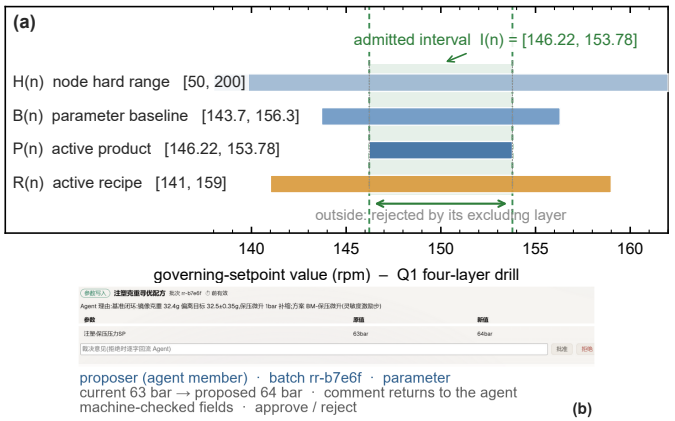


Fig. 3. The admission boundary and its human-approval gate, on the cast-film reference line. (a) The Eq. (1) interval geometry: four declared, versioned bounds narrow to the admitted interval; a proposal outside it is rejected by its excluding layer (the Q1 four-layer drill). (b) The structured approval card on a manual binding (live system, gloss added): machine-checkable fields are separated from the proposer's display-only reasoning; the comment returns to the member on refusal (Section IV-E).

every step leaves its own evidence record, while execution and recovery are deliberately not atomic with device state (Section VI). Six invariants organize the path—binding-resolved authority (never channel membership or prompt content), Eq. (1) admission, fail-closed approval waits on manual bindings, graded rather than success-flag acknowledgment, checkable three-way consistency, and separately admitted compensation—each specified in the subsections that follow, and every agent-originated path to the plant passes through the same governed write.

A. Task, Observation, and Admission

The strategy receives a task and goal in production context and decides what to propose and when to

stop; the platform governs execution over semantic node parameters—pressure, dosing rate, line speed, rail width—in engineering units.

For an ordinary agent setpoint request on node n, the admission interval is

$$\mathcal{I}_n(t) = \mathcal{H}_n \cap \mathcal{B}_n \cap \mathcal{P}_n(t) \cap \mathcal{R}_n(t), \tag{1}$$

where $\mathcal{H}_n$ is the mandatory hard engineering range (finite and non-empty at node creation) and the remaining terms the baseline, active-product, and active-recipe limits; a missing optional limit contributes no restriction, so the interval is always defined, and t is the admission instant, re-resolved at execution. Should a configuration empty the intersection, every write is rejected with the excluding layer named, surfacing the misconfiguration at the first write. The interval form is deliberate: every bound is a declared, versioned object that an audit can replay against the active product and recipe, and rejection is decidable and layer-attributable, where a learned feasibility model offers neither (Fig. 3a)—a decidability claim, not a safety certificate. It bounds one parameter at a time—joint, cross-variable windows live in scenario guards—and bounds magnitude, not intent: availability, binding state, and approval policy are checked separately, and in-window but adverse sequences surface at verdict and compensation (Section V-F reports two).

B. Semantic Write, Pacing, and Source-Specific Routes

A semantic card exposes quantity, unit, precision, range, value, freshness, and recipe context; the controller resolves binding and limits, the driver serializes the command, and the acknowledgment grade closes the command path, never the process-response path (Section IV-C). Three pacing windows pace governed loops, never the plant: online writes—agent- or human-origin—sit at least 60 s apart with a hold window after success (HTTP 429 inside it); a per-recipe dispatch interval anchored at the approval instant is restart-persistent, so alternating identities cannot reset it; and a 300 s per-line trial cadence damps same-line trials from any origin. Recipe-dispatch and rollback writes are exempt. AW_BENCH_MODE=1 admits harness-declared arms that relax the soft window or readback—never the hard range.

Four governed routes—agent tools, human-origin writes, recipe dispatch, and rollback compensation—reach the same driver through one admission and journaling entry point, the hard range included; recipe dispatch adds run context—the line-scoped execution window, the reduced ISA-88 batch/lot notion—validated at save time. Hold heartbeats (periodic re-dispatch of the stored setpoint) are the one non-governed re-dispatch: off by default, stopped by line stop or driver change, and re-applying the stored value without re-admission—plant-facing state maintenance, not an agent path. Line start installs active context before recipe parameters dispatch individually, and line stop clears it; these multi-step operations are not atomic with device state.

C. Write Acknowledgment as Three-Level Evidence

A driver-level success flag conflates two facts: the transport accepted the command, and the device now holds the value. AgentWorkShop grades every governed write into one of three acknowledgment levels: readback-verified when an independent read—one issued outside the write path—returns the commanded value within the node's configured write tolerance; transport-acknowledged when the transport accepted the command on an endpoint with no usable readback; unverified when the transport failed or when post-write verification could not confirm the value. The grading rule is falsifiable. Under a success-flag receipt rule—a transport acknowledgment closes the report—the publish-only receipt demonstrated in Section V-G (broker puback, ok=true, no read channel, three verification attempts) reports success; the graded rule reports it unverified. That is what fail-honest means: absence of evidence degrades the grade, never the report, and the grade is fixed when the write closes.

A transport-acknowledged write may still not have reached the device, so a post-write verifier re-reads the endpoint (three attempts, 2 s apart) and promotes the grade when a read confirms the value. Batch receipts carry a per-parameter grade plus a summary; the write journal stores the per-write grade, and the operations log records the verdict at worst-state severity. Recipe rollback restores a definition, not physical PLC values, and a compensating write is itself graded by the same three levels. The grade closes the command path only: readback-verified says the device holds the value, not that the process responded as intended—that distinction belongs to observation and verdict.

D. Observation, Verdict, and Compensation

An agent- or rollback-source write opens an optimization record; the verdict tool dcw_judge records keep, rollback, or uncertain plus a reason; the verdict, stored with the record it closes and the response window actually queried, performs no actuation and does not independently validate the evidence, so a scripted keep can coexist with a missed objective. Verdict integrity rests on attribution, journaling, and operator review: a kept-but-marked record stays visible with its disagreement attached, and the operations log raises parameter severity under alarm-rationalization practice. Three-way setpoint consistency is checked by the evaluation harness rather than the platform: the harness compares the recipe value, the device readback, and a mirror sample drawn from the acquisition runtime's separate polling path and store, each leg against a stated tolerance; the sampled legs meet at the plant endpoint, so a fault fabricating the endpoint is a common-mode boundary, and disagreement is reported with the run. Verdicts are attributable—actor and timestamp travel with them, and corrections amend the stored reason visibly.

Compensation is a new, separately admitted write that may fail or partially complete; it is graded by the Sec-

tion IV-C levels, and a rollback record settles to rolled back once the compensating write is accepted—readback-verified where the endpoint supports readback, transport-acknowledged otherwise. If the compensating write is itself refused by admission, the source record closes as judged, with the refusal appended to its verdict. These separations keep command status, record status, and process recovery independently inspectable.

The backstop is a record-integrity check for writes that no verdict has closed—not an independent protection layer: it evaluates only open records; a caller's keep closes the record and stands it down, while a rollback verdict stands the backstop down only once the compensating write executes—availability is therefore contingent on recorded verdicts and arriving observations. When collected samples of an open record's node leave the recipe's monitoring window (a DAQ-side envelope distinct from the admitting interval), the platform issues a system-attributed rollback verdict and restores the value preceding that write (one step; the admitted baseline is not restored), graded as above. The policy waits 120 s, polls every 30 s, and fires on three out-of-window samples—cumulatively, not necessarily consecutively; non-returning samples count no breach, so an acquisition outage leaves the record open and unalarmed. Recovery runs only while fewer than two retained rollback records stand on the node, past which the backstop judges but defers actuation to the governing approval policy.

E. Human Approval Boundary and the Line Master Switch

Manual bindings insert the timed approval wait before an agent setpoint or compensation request; automatic bindings omit the wait but keep the checks. The pending request shows member, node, value, interval, and expiry—plus the proposer's reasoning and per-parameter basis (Fig. 3b): machine-checked fields are structured data, the free-text reasoning display-only and non-authoritative. Because binding and admission are re-checked at execution, approval reserves neither interval nor device state; the approval dialog—itself an injection surface (Section VI)— cannot promote a proposal past the checks; the comment returns verbatim to the proposing member on refusal and travels with the receipt on approval—feedback, not authority. The wait fails closed by default (expiry 180 s): an expired request resolves as refusal and is recorded; duplicates are suppressed by holding one pending request per member–node pair; and in hold mode an unanswerable request persists without expiry until a human answers, repeat requests refused as duplicates.

Human authority is anchored one level above individual requests: each line carries a control mode—the line master switch—whose fail-safe default is manual. Under manual mode, recipe-family operations (parameter update, dispatch, trial, and rollback) from approval-free bindings on that line are intercepted and presented for approval regardless of binding policy; direct node-tool writes remain gated by their binding mode and the admission interval, not by the line switch. Switching a line to automatic requires an explicit, admin-only user-side confirmation—agent identities cannot touch the line switch. The residual agent-side path is delegation: a lead agent can grant held nodes to a worker through an attributed lead-only tool; the grant's execution mode is chosen at grant time rather than gated by the user-side confirmation, is attributed to the granting agent, cannot exceed its own node face, and is individually revocable (Section VI).

The decision endpoint authenticates a user but does not enforce a line-scoped approver role—the gap Section VI carries—and reads, verdict recording, and some configuration changes do not use physical-write approval—workflow mediation, not segregation of duties; a recipe switch between approval and execution re-admits the value against the new window (Section VI). The strategy engine is replaceable rather than trusted as a control primitive: scenario code declares and configures but never executes plant I/O, and the platform never infers a process objective.

V. Experimental Evaluation

The evaluation asks whether one governed supervisory path can be exercised, inspected, and repeated across heterogeneous industrial jobs, not whether one optimizer beats another. Five questions organize the evidence: protocol closure and admission-gate attribution (Q1), the agent-team workflow (Q2), a deterministic controller loop (Q3), four concurrent missions (Q4), and deployment-scale regression with write-evidence verification (Q5). Q2–Q4 occupy the strategy slot (scripted policies, with two live engine turns in Q2); Q5 stresses the hardened platform under load. Together the five exercise each production-facing plane of Section III.

A. Testbed, Evidence Snapshot, and Scope

The evaluation draws on two evidence tiers of one codebase. The archived snapshot for Q1–Q4 is integrated run 20260921184204-10lc (seed 42, commit e692df1, harness 9f2756dce4a4cbb3, completed September 21, 2026, 19:08 UTC), archived with raw reports, plus ablation companion 20260921181234-e1lite. Its standing preset is a cast-film reference line tuned through the die-head heater setpoint (objective J weights thickness conformance, defects, energy, and throughput under melt-temperature and pressure constraints, larger better). Each mission contributes one seed-42 trajectory: Q1, Q2, and Q4 are single-trace case studies, checkable claim by claim in the archived artifacts; the deterministic loop adds seeds 43–44, and repeat evidence comes from Q5's consecutive deployment-tier campaigns—a different tier, not a re-run.

The deployment-scale tier for Q5 is a live v0.7.57 instance carrying 34 provisioned lines, 112 agent channels, 67 online DAQ nodes, 77 write nodes, 52 recipes, and 385 member–node bindings—simulator- and bench-backed objects, not physical plants; multi-line means many

provisioned lines and channels on one host. The tier's benchmark line—a different plant from that cast-film reference—is a physics-based injection-molding cell (16 DAQ nodes over the four modalities, 6 control nodes over five protocol families plus MES, four exercised for governed writes); all Q5 numbers come from committed per-run reports, ledgers, and timelines (October 7–8: hardened-pipeline run -20261008045434 plus three consecutive full-suite runs; October 9: benchmark-20261009021149/-024141/-033810); mechanisms are unchanged since the Q1–Q4 snapshot except the October hardening pass's acknowledgment grading and master switch; the campaign ran under live load. The harness sets the benchmark line's master switch to automatic by user-side confirmation—the automated path is the object under test—and verifies the switch's manual-mode interception separately (Section V-G, leg 1).

B. Test Procedure and Campaign Overview

The Q1–Q4 procedure is a 20-phase integrated suite—an author-defined regression instrument, not an external certification—probing one claim per phase, from preset loading against the best grid point ($J^* = 89.894$) through protocol interception and the record, recipe, approval, and parameter-layer drills (Sections V-C and V-F).

The Q5 procedure is the one-command benchmark pipeline: eight stages (S0 pre-check; S1 API-surface matrix, a positive-and-negative sweep across the functional surface of Sections III–IV; S2 multi-modal ingestion with MES dual-mode; S3 governance negative paths; S4 closed-loop optimization; S5 fine-tuning; S6 diagnostics; S7 report) under a harness that captures every check and a timestamped event timeline per run; the October 9 campaign re-runs it three times. No prior system in Table I owns a governed write path to plant endpoints, so the primary comparator is internal, and two controls are archived. The admission control holds the write path fixed and relaxes its mechanisms: the six window-edge probes that full governance rejects (HTTP 400) return 200 executed under the relaxed arm (Section V-C, Table II)—a success-flag design reports them as accepted writes. The evidence control holds the receipts and varies the reporting rule (Section IV-C); the success-flag outcome is definitional—the pre-hardening harness itself reported a broker puback as success. The closed-loop harness also parameterizes the write injection (admin REST beside the agent's governed dcw_control); on the shared live instance, a like-for-like pair (four archived post-campaign runs, October 12) saw every write rejected in full on both routes—24/24 writes per arm under production pacing, and 24/24 again with pacing exempted; the archived runs record aggregate rejections only, and a closed-loop quality comparison awaits a dedicated instance. The harness's own reports verdict these attempts fail; they are archived as gate evidence. Fig. 4 summarizes the closed-loop instrument.

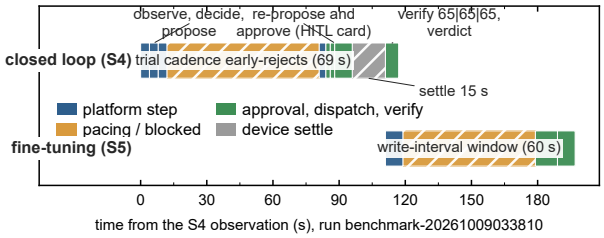


Fig. 4. One governed iteration on the October 9 line (harness-adjudicated approval): a corrective decision (0.276 g error) packaged by the proposer as a sensitivity excitation; the trial cadence early-rejects 69 s before a human is involved; structured approval releases the re-proposal, dispatch returns readback-verified, and the three-way check reads 65|65|65. Lower lane: fine-tuning under the same 60 s window, rolled back and restored.

C. Q1: Protocol Closure and Admission-Gate Attribution

On the five protocol lines the suite stores 55 DAQ samples through real drivers and dispatches governed writes over four protocol families—about 6 writes each, p50/p95: 47.8 ms/248.9 ms (Modbus TCP), 19.8 ms/20.5 ms (OPC UA), 19.3 ms/20.1 ms (MQTT), 32.9 ms/44.6 ms (HTTP); the RTU-over-TCP satellite is acquisition-only—recording readback confirmation where supported—with one admission, record, and verdict path serving every request. Each writable line rejects 6/6 out-of-window probes (24/24 intercepted) while accepting its legal write; the window-interlock drill probes the window edge and both admission branches. Two drills separate truth paths one success flag would collapse: a frozen-measurement drill (write and readback consistent, measurement out of window) ends with a system-judged backstop rollback 130.2 s later, and a record-rollback drill restores the prior value by a separately admitted, readback-confirmed write. A second preset with different signals and dynamics is commissioned by configuration alone; across both presets 33 probes are rejected on seven writable lines—24 on the first preset's four writable lines, 9 on the second preset's three—the same admission code discriminating on unseen signal maps. The parameter layer makes the interval geometry of Eq. (1) observable (Fig. 3a): a four-layer chain—node $[50, 200]$, parameter $[143.7, 156.3]$, product $[146.22, 153.78]$, and recipe $[141, 159]$ rpm—narrowing to one admitted interval, each rejection attributable to its excluding layer; a manual binding suspends a request visibly and unblocks it on approval (latency 461 ms, harness-adjudicated and poll-dominated; Q5 reports the server-side span alone).

The ablation replays the same probes on companion run 20260921181234-e1lite and a standalone fixture (six probe classes per arm, three deterministic repetitions with identical counts); it isolates the window interlock, untouched by the October hardening pass. Table II reports the four arms: full governance and the no-readback arm reject 18/18; without the window interlock (alone or with the readback check) only 12/18 reject and the six window-class probes execute and are journaled—prevention is the interlock's job; a relaxed configuration degrades from

TABLE II
Admission ablation on companion run 20260921181234-e1lite: six probe classes per arm, three deterministic repetitions. The four hard-range classes are rejected in every arm; only the two window-edge classes flip.

Arm	Soft win.	Readback	Intercept	Executed
full	✓	✓	18/18	0
no-readback	✓	—	18/18	0
no-interlock	—	✓	12/18	6 jour.
ungated	—	—	12/18	6 jour.

prevention to detection, with the readback check retained to make written-but-unconfirmed commands visible. The hard range is structurally unbypassable—all four governed routes re-admit it at one journaling entry point, per request (Section IV-B)—and the soft window is its removable part.

D. Q2: The Agent-Team Optimization Workflow and Its Results

The workflow ran end to end on the cast-film line and then on a larger BOPET line (Section III-B); a scripted policy rather than a language model issued the writes (two live engine turns follow). On the single-node mission one write sufficed ($200 \rightarrow 204.704\,^{\circ}\mathrm{C}$ against target $204.704 \pm 0.75\,^{\circ}\mathrm{C}$, settled at $204.70\,^{\circ}\mathrm{C}$, journal-attributed, task COMPLETED). The BOPET mission is the multi-node case: nine devices and 49 signals (30 writable, 19 readable), all ensure-pass created. Three governed writes on three distinct nodes—casting speed $32 \rightarrow 33.8$ m/min, fast-roll speed $118 \rightarrow 120.3$ m/min, and rail width $3000 \rightarrow 3035$ mm— bring thickness from 28.10 to 25.66 μm against $25.0 \pm 0.7\,\mu$m—0.04 μm inside the window; the same run's prose summary records it not attained while its machine-readable verdict records attained, so we report both; the verdict matches the archived predicate—the prose narration erred. Each write opened a record, received keep, and left an agent-attributed journal entry, the parent task reaching COMPLETED; governed-write counts match accepted changes in both missions. Finally, two live agent-engine turns (omp adapter, glm-5.3-flash) drove the same slot on a scratch line—a bounded write (156.3 s) and a goal-driven multi-step descent settling at 51.7 μm against a harness-supplied target ($52.0 \pm 0.8\,\mu$m written into the task text, not the scenario predicate), and saves a recipe with attributed authorship (637.3 s). They show that an engine can occupy the slot, not agent quality: the run's pass criterion is a completion marker in the transcript.

E. Q3: The Deterministic Closed Loop

Seeds 42–44 each issue two governed writes with no rejection, raising the cast-film objective J from $J_0 \approx 67.2$ toward its grid optimum ($J/J^* = 0.965$–0.972 against $J^* = 89.894$)—an exhaustive grid search over the admissible setpoints (zone temperature, screw speed, and line speed over their declared ranges) on the same simulator, so J^* is a lower bound on the true optimum and the reported J/J^* an upper bound on J/J_{true} ($J > 0$ throughout): the true gap is at least as large as shown. What the benchmark

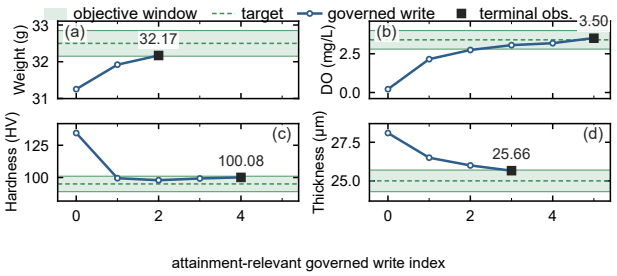


Fig. 5. Archived optimization trajectories of the four concurrent missions (run 20260921184204-10lc, seed 42): observed controlled quantity after each attainment-relevant governed write, against the archived objective window; the index counts those writes, so the 6 plotted points of (b) are 6 of its 15 governed writes; (a) injection molding, (b) wastewater treatment (A2O), (c) annealing, (d) BOPET film. DO, dissolved oxygen. Simulator observations under scripted policies.

establishes is narrower: the governed path passes a conventional controller's writes through unmodified—admissible, journaled, read back. The claim is transparency of evidence handling, not a performance delta.

F. Q4: Four Process Missions

The campaign runs four blueprint-provisioned lines concurrently; in this run (seed 42) all four attain their archived predicates—evaluated from subsequent observations, not task state. Fig. 5 archives each mission's optimization trajectory. Injection molding is the minimal case (two writes: hold pressure 45→58.2 bar, hold time 6→10.64 s), BOPET the multi-node case of Section V-D; both close with task state and predicate agreeing, on separate records. Wastewater treatment (anaerobic–anoxic–oxic) is the write-heaviest mission (15 governed writes): dissolved oxygen rises from 0.21 into 3.4 ± 0.6 mg/L (attainment window; terminal observation 3.50 mg/L, Fig. 5b) under five effluent guards: four guard margins finish wider than at baseline, and the fifth, pH, drifts 6.59→6.38. Its most informative events are blower steps that were admissible and journaled yet lost compliance margin in the post-write observation; the verdict stage reverted them, so a bad proposal is rejected in the evidence, not at the admission gate. Annealing starts far outside the hardness window (134.52 HV at 140 m/min); zone-3 temperature, one zone-2 trim, and two line-speed steps hold 95 ± 6 HV while raising line speed by 17.1%, and a further attempt at 176 m/min is reverted for leaving the margin—on record. Mission clocks (26.48 s, 169.27 s, 102.76 s; BOPET's unarchived) are simulated mission durations under accelerated dynamics. Across the four missions in this run, platform-side behavior is identical while the write economy—governed writes per scripted policy—varies by more than a factor of seven (2, 15, 5, and 3).

G. Q5: Deployment-Scale Regression and Write-Evidence Campaign

The final lens stresses the platform itself: the hardening pass made the evidence grading, the line master switch, and the approval-anchored dispatch interval of Section IV

the explicit test surface of the closed-loop, fine-tuning, and write-evidence scenarios. Full-suite repeatability. Seven pipeline executions complete with zero failed checks—79 checks per run on October 7 (237 total), 91 on the hardened-pipeline run, 90–92 per run on October 9 (600 total); two October 7–8 runs land inside the per-line trial cadence and wait out the remaining 218 s— pacing stays in force under automation.

Closed-loop scenario under governance. Each run walks observation, grey-box-sensitized decision (dead-band 0.15 g, slope 0.052 g/bar), a five-element per-parameter proposal, HITL approval, batch dispatch, and three-way consistency (recipe, PLC readback, mirror sample). The loop runs in two regimes: October 7–8 descends $66 \rightarrow 63$ bar one governed step per run under converged dead-band excitation; October 9 answers a sagging mirror weight (32.49/32.33/32.22 g) with a bounded zigzag (64↔63 bar, converged) and corrective steps (63↛65 bar). The zigzag is a deliberate excitation, not a correction, while the two corrective steps answer residual error (0.169/0.276 g). Every step verifies equal values across the three sources at maximum consistency error 0.0 bar (0.05 bar device, 0.51 bar bucketed-mirror tolerances—all three legs equal exactly at the endpoint's recording resolution). The pacing window acts before the human: in two October 9 runs the proposal is early-rejected by the trial cadence and re-proposed 69 s later; only then is the HITL card raised (scheme, proposer reasoning, per-parameter basis) and structured approval releases the readback-verified dispatch (Fig. 4). Approval-to-dispatch completes in 6–9 ms; observation-to-verification spans 25.1 s without a pending cadence window (245.20/245.18 s with, October 7–8; 96.1 s, October 9)—the loop's time is governance, not platform overhead. The fine-tuning scenario rides the same windows (Fig. 4, lower lane), and the diagnostics scenario exports a 30-min merged wide table (663–869 rows, 14 columns)—a feed data-quality observation, not a process excursion.

Latency and mission figures are single-run point estimates (about 6 writes per line, one trajectory each); only Q3 and Q5 repeat across seeds, while the Q1 ablation repeats deterministically (Table II).

Write-evidence legs. A dedicated seven-leg suite verifies Sections IV-C and IV-E directly: (1) the master switch refuses an unconfirmed manual-to-automatic transition; (2) a mock-acknowledged dispatch is promoted to readback-verified; (3) a broker cut marks affected parameters unverified while healthy siblings verify; (4) a publish-only endpoint accepts the write (broker puback, ok=true) yet stays unverified through three verification attempts—the receipt Section IV-C's rule is stated against; (5) on a manual-mode line an approval-free binding's dispatch is intercepted into the queue, approval releasing all six parameters device-verified; (6)–(7) API regression and documentation-consistency suites close the campaign.

Across the five questions, Section IV's mechanisms behave as specified: interception with attribution, honest grading, pacing under automation, mediated approval, and on-record recovery.

VI. Discussion and Limitations

A. Industrial-Informatics Interpretation

Against the nearest systems of Section II, the differentiator is not the LLM—a replaceable strategy slot—but the production-scoped authority and evidence around its writes: no nearest system reports that combination on one path. Deployment-wise, the layer installs as one service beside the plant's protocol endpoints; onboarding is declarative (blueprint, differential ensure pass, team-template channel, mission guards), approval clients—web panel or API—receive the HITL cards, and PLC protection and DCS interlocks stay untouched.

B. Validity and Reproducibility

Run identifier, seed, commit, harness fingerprint, and raw artifacts are fixed, so counts are re-derivable rather than trusted; Q5 adds per-run ledgers and timelines. Five qualifications bound what they support: simulation-tier evidence (deterministic per seed; no physical-device campaign, operator study, or external comparison reported); internal-oracle optima and predicates; single-host latencies and near-contemporary runs; implemented-but-unexercised capabilities and a single live provider/model; and harness-adjudicated approval (human decision latency and quality out of scope).

C. Safety Positioning and Human Oversight

The threat model is a misbehaving or manipulated agent (wrong or hallucinated proposals) inside an otherwise trusted platform and plant; the approval dialog, admission re-checks, and journaling are the containment surfaces; no adversarial injection campaign or approval-dialog operator study is reported; stolen credentials and compromised endpoints are the domain of IEC 62443 [18]. Fast regulation, hardware interlocks, and safety functions remain in the PLC; prompts and retrieved documents are untrusted text, not authority [24], [25]—the proposer is confined to per-binding scope and journaled, attributable actions: a bounded blast radius, not a protection guarantee. Admission limits, acknowledgment grading, and the backstop run with or without human approval—enforcement never queues behind human attention. The pacing windows bound approval demand per line, and batch release (Section V-G, leg 5) is the demonstrated fatigue mitigation; approval fatigue and automation bias remain human-factors risks, and reading graded evidence assumes trained approvers [23], [26]. A lead agent's delegation-mode choice (Section IV-E) is the one agent-reachable path around per-request approval; capping granted mode at the granter's own binding mode, or requiring the line-switch confirmation for it, is the queued guard.

D. Deployment Boundaries

Execution and recovery are non-atomic: bookkeeping may fail after actuation, the pacing windows time writes rather than certify stability, declared per-write step limits are advisory metadata surfaced to agents rather than enforced bounds, and multi-node compensation may partially succeed. Readback agreement and three-way consistency need not imply process recovery, and hold heartbeats re-apply the stored setpoint without re-admission, trailing a narrowed window. Oscillation, storage failure, clock error, concurrency beyond the four-line campaign, and redundant supervisor deployments with divergent stores (split-brain) remain unevaluated; a supervisor outage halts dispatch rather than permitting writes through, and a hardware-in-the-loop leg against a real PLC or OPC-UA endpoint is the next evidence tier. Production integration additionally requires: bound changes managed under change-control with process-evidence validation; approval-aging, open-record-aging, and backstop-deferred states surfaced on the plant alarm system; the bench-mode environment switch refused at production boot; and approval treated as workflow mediation—attribution and timestamp, not an electronic signature. In production, bindings govern sanctioned tools, not arbitrary privileged-engine I/O—least privilege, endpoint security, and IT/OT separation remain necessary [18]; AgentWorkShop is neither a safety instrumented function nor a plant-protection substitute [27]; line-scoped approver authorization is still missing, so production needs segregation of duties and alarm-path failure tests [28].

VII. Conclusion

AgentWorkShop contributes a governed supervisory execution layer rather than a new process optimizer: one stable path from agent-team identities through semantic nodes, production context, governed writes, graded acknowledgments, verdicts, and compensation, with admission, approval, and recovery left as separately inspectable, actor-attributed evidence on every agent-originated write. Physical-plant deployment, operator studies, and external-framework comparison remain open; until then, optimization benefit, autonomous control, and safety assurance stay unsupported claims.

Data availability and disclosure. Artifacts are archived per run in the project's benchmark result tree; anonymized artifacts are available during review and released with the code upon publication. The two live engine-turn console transcripts (Section V-D) were excluded by the repository's log-retention rule and survive only as recorded outcomes; the write-evidence suite's checks are documented in the audit report and rerunnable from its script. AI assistants supported editing under author direction; the authors remain responsible for all claims.

References

[1] K.-C. Huang, H.-T. Lin, C.-H. Lin, and M.-T. Huang, "Large language model integration in industrial cyber-physical systems: A multi-modal architecture for automated control generation," in Proc. Int. Conf. on Control, Automation and Diagnosis (ICCAD), 2026, pp. 1–6.

[2] K. Köhle, D. Anicic, T. A. Runkler, and R. Graf, "LLM-driven multi-agent control for skill-based smart manufacturing," in Proc. IEEE 31st Int. Conf. on Emerging Technologies and Factory Automation (ETFA), 2026.

[3] J. Lim and I. Kovalenko, "A large language model-enabled control architecture for dynamic resource capability exploration in multi-agent manufacturing systems," in Proc. IEEE 21st Int. Conf. on Automation Science and Engineering (CASE), 2025.

[4] Z. Zhao, D. Tang, C. Liu, L. Wang, Z. Zhang, H. Zhu, K. Chen, Q. Nie, and Y. Ji, "A large language model-based multi-agent manufacturing system for intelligent shopfloors," Adv. Eng. Inform., vol. 69, 2026, art. no. 103888.

[5] M. Waseem, K. Bhatta, C. Li, and Q. Chang, "Pretrained LLMs as real-time controllers for robot operated serial production line," Mach. Learn.: Eng., vol. 2, no. 1, 2026, art. no. 015006.

[6] Z. Liu, R. Zeng, D. Wang, G. Peng, X. Liu, Q. Liu, P. Liu, W. Wang, and J. Wang, "Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents," IEEE Trans. Softw. Eng., vol. 52, no. 5, pp. 1672–1687, 2026.

[7] P. Leitão, "Agent-based distributed manufacturing control: A state-of-the-art survey," Eng. Appl. Artif. Intell., vol. 22, no. 7, pp. 979–991, 2009.

[8] A. Biskupovic, A. Villalonga, F. Castaño, R. E. Haber, and F. Núñez, "Agent-based supervision for service-oriented industrial cyber–physical systems," IEEE Trans. Ind. Inform., vol. 21, no. 3, pp. 2719–2728, 2025.

[9] X. Jiang, H. Xie, J. Wang, Z. Yang, Y. Zhou, L. Yao, and Z. Zhu, "Agentic AI for safety-aware process monitoring and fault diagnosis: A review," Processes, vol. 14, 2026, art. no. 2112.

[10] Anthropic, "Model context protocol," [Online]. Available: https://modelcontextprotocol.io, 2024, accessed: Sep. 20, 2026.

[11] International Society of Automation, "ANSI/ISA-88.00.01-2010—batch control—part 1: Models and terminology," Standard, 2010.

[12] ——, "ANSI/ISA-95.00.01: Enterprise-control system integration—part 1: Models and terminology," Standard, 2010.

[13] J. Moyne, E. del Castillo, and A. M. Hurwitz, Run-to-Run Control in Semiconductor Manufacturing. Boca Raton, FL: CRC Press, 2000.

[14] Z. Xiang et al., "GuardAgent: Safeguard LLM agents by a guard agent via knowledge-enabled reasoning," 2024, arXiv preprint arXiv:2406.09187.

[15] U.S. Food and Drug Administration, "21 cfr part 11: Electronic records; electronic signatures—scope and application," Regulation guidance, 2003.

[16] International Society for Pharmaceutical Engineering, "Gamp 5: A risk-based approach to compliant gxp computerized systems," Standard practice guide, 2022.

[17] NAMUR, "Namur open architecture (noa)—second-channel integration for automation components in the process industry," Industry specification (Worksheets NE 175 series), 2017.

[18] International Electrotechnical Commission, "IEC 62443-3-3:2013: Industrial communication networks—network and system security—part 3-3: System security requirements and security levels," Standard, 2013.

[19] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in Proc. AAAI Conference on Artificial Intelligence, vol. 32, no. 1, 2018.

[20] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," IEEE Trans. Autom. Control, vol. 62, no. 8, pp. 3861–3876, 2017.

[21] L. Sha, "Using simplicity to control complexity," IEEE Softw., vol. 18, no. 4, pp. 20–28, 2001.

[22] C. M. Crowe, "Data reconciliation—progress and challenges," J. Process Control, vol. 6, no. 2–3, pp. 89–98, 1996.

[23] L. Bainbridge, "Ironies of automation," Automatica, vol. 19, no. 6, pp. 775–779, 1983.

[24] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in Proc. 16th ACM Workshop on Artificial Intelligence and Security, 2023, pp. 79–90.

[25] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in Proc. 33rd USENIX Security Symposium (USENIX Security 24). USENIX Association, 2024, pp. 1831–1847.

[26] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," IEEE Trans. Syst., Man, Cybern., Part A, vol. 30, no. 3, pp. 286–297, 2000.

[27] International Electrotechnical Commission, "IEC 61511-1:2016: Functional safety—safety instrumented systems for the process industry sector," International Standard, 2016.

[28] International Society of Automation, "Ansi/ISA-18.2: Management of alarm systems for the process industries (iec 62682)," Standard, 2016.